

Actividad Guiada: Funciones Definidas por el Usuario en PostgreSQL

Una empresa de ventas necesita automatizar ciertos cálculos que se repiten constantemente: aplicar impuestos, calcular descuentos y obtener reportes de ingresos de clientes. En lugar de repetir fórmulas en cada consulta, han decidido crear funciones definidas por el usuario (UDFs) en PostgreSQL para simplificar y estandarizar la lógica.

Ejercicio Autoinstruccional

Enunciado del reto

Tu reto es diseñar, implementar y probar al menos dos funciones SQL que den solución a estos requerimientos, aplicando los contenidos vistos en clase sobre funciones definidas por el usuario.

Objetivo(s) de la actividad

- Identificar los componentes y casos de uso de una función en SQL.
- Implementar funciones definidas por el usuario (SQL Functions) en PostgreSQL.
- Aplicar funciones a consultas sobre tablas reales.
- Validar que las funciones resuelven un problema de negocio de forma correcta.

Materiales y preparación previa (checklist)

- Acceso a PostgreSQL (pgAdmin o cliente equivalente).
- Lectura base: Funciones definidas por el usuario en SQL.
- Base de datos de ejemplo: tabla ventas con al menos los campos id, cliente, producto, cantidad, precio_unitario, fecha.

Requisitos técnicos y datos

- Motor PostgreSQL.
- Permisos para crear funciones y ejecutar queries.
- Conocimientos previos: sentencias básicas CREATE TABLE, INSERT, SELECT, GROUP BY.

Instrucciones Paso a Paso

01

Exploración inicial (5 min)

Revisa la definición de una función en SQL (nombre, parámetros, tipo de retorno, cuerpo, lenguaje).

Micro-check: escribe un esquema básico en tu cuaderno:

```
CREATE FUNCTION nombre_funcion(parámetros)
RETURNS tipo
AS $$
SELECT ...
$$ LANGUAGE SQL;
```

03

Función con aplicación a tabla (10 min)

Crea una función llamada `ingresos_cliente` que reciba el nombre de un cliente y retorne la suma total de sus compras.

Aplica la función sobre la tabla `ventas` con registros de ejemplo.

Ejemplo de prueba:

```
SELECT ingresos_cliente('Juan Pérez');
```

Micro-check: confirma que el total coincide con la suma manual de las ventas de ese cliente.

02

Función de cálculo simple (10 min)

Crea una función llamada `calcular_total` que reciba cantidad y precio_unitario y retorne el valor total de la venta.

Ejemplo de prueba:

```
SELECT calcular_total(5, 120); -- Resultado esperado: 600
```

Micro-check: valida que el resultado sea correcto para distintos valores.

04

Reflexión y extensión (5 min)

Explica en un párrafo cómo el uso de funciones mejora la reutilización, seguridad y eficiencia de las consultas en la empresa.

Micro-check: compara una query repetida con la misma lógica vs. el uso de la función.



Tiempo total: 30 minutos

Criterios de Éxito y Errores Comunes

Criterios de éxito

- Se definen correctamente los componentes de cada función (nombre, parámetros, tipo de retorno, cuerpo, lenguaje).
- Las funciones creadas son ejecutables y reproducibles en PostgreSQL.
- Los resultados obtenidos corresponden con los cálculos esperados.
- Se entrega una breve justificación escrita del beneficio de las funciones.

Errores comunes

- Olvidar LANGUAGE SQL.
- Declarar mal el tipo de retorno (NUMERIC vs INT).
- No probar las funciones con valores distintos.

Archivo Solución (SQL)

Creación de tabla y datos de ejemplo

```
-- Creación de tabla de ejemplo
CREATE TABLE ventas (
  id SERIAL PRIMARY KEY,
  cliente TEXT,
  producto TEXT,
  cantidad INT,
  precio_unitario NUMERIC,
  fecha DATE
);

-- Insertar datos de prueba
INSERT INTO ventas (cliente, producto, cantidad, precio_unitario, fecha) VALUES
('Juan Pérez', 'Laptop', 2, 1200, '2023-01-15'),
('Ana Gómez', 'Teléfono', 1, 800, '2023-02-10'),
('Juan Pérez', 'Tablet', 3, 500, '2023-03-05');
```

Funciones implementadas

```
-- Función 1: calcular total de venta
CREATE OR REPLACE FUNCTION calcular_total(cantidad INT, precio NUMERIC)
RETURNS NUMERIC
AS $$
SELECT cantidad * precio;
$$ LANGUAGE SQL IMMUTABLE;

-- Prueba de función calcular_total
SELECT calcular_total(5, 120); -- Esperado: 600

-- Función 2: ingresos por cliente
CREATE OR REPLACE FUNCTION ingresos_cliente(nombre_cliente TEXT)
RETURNS NUMERIC
AS $$
SELECT SUM(cantidad * precio_unitario)
FROM ventas
WHERE cliente = nombre_cliente;
$$ LANGUAGE SQL STABLE;

-- Prueba de función ingresos_cliente
SELECT ingresos_cliente('Juan Pérez'); -- Esperado: 2*1200 + 3*500 = 3900

-- Función opcional: ventas en rango de fechas
CREATE OR REPLACE FUNCTION ventas_rango(fecha_inicio DATE, fecha_fin DATE)
RETURNS TABLE(cliente TEXT, total NUMERIC)
AS $$
SELECT cliente, SUM(cantidad * precio_unitario) AS total
FROM ventas
WHERE fecha BETWEEN fecha_inicio AND fecha_fin
GROUP BY cliente;
$$ LANGUAGE SQL STABLE;

-- Prueba
SELECT * FROM ventas_rango('2023-01-01','2023-12-31');
```